UNIVERSIDAD MARIANO GÁLVEZ

SEDE: CHIMALTENANGO, COLEGIO MESOAMERICANO

FACTULTAD DE INGENIERÍA EN SISTEMAS

CURSO: 027 – BASES DE DATOS I

INGENIERO HÉCTOR JAHAZIEL LUNA IRRIATE



DOCUMENTACIÓN BACK-END — PLATAFORMA DE APUESTAS DEPORTIVAS

FECHA: 31 DE OCTUBRE DE 2026

AJUCHÁN POPOL, ALLAN ANTHONY – 1990-24-13391

GIRÓN MORALES, OSCAR EDUARDO – 1990-24-12952

MOX SUJUY, DIEGO JULIO ALBERTO – 1990-24-14979

PICHIYÁ APÉN, JOSÉ NEHEMÍAS – 1990-24-25116

TZUNUX TUM, JACAWITZ IXCHEL – 1990-22-11512

Documentación Back-end — Plataforma de Apuestas Deportivas

1. Introducción

La Plataforma de Apuestas Deportivas y Pronósticos es una aplicación web en Django que gestiona usuarios, billetera virtual, eventos deportivos, boletos de apuesta y registros de seguridad, con SQL Server como base de datos.

Este documento explica las tecnologías del back-end y cómo el proyecto usa la base de datos. Se basa en settings.py, el script SQL, las migraciones, los models.py, urls.py, views.py y forms.py de las apps, las plantillas, las utilidades de autenticación y los dos middleware.py propios. Lo que falta por revisar está en la sección 10.

2. Arquitectura general

El proyecto sigue el patrón MTV de Django (modelo, plantilla, vista) y se divide en seis apps, cada una con una responsabilidad.



recorrido de una petición · 2 middlewares propios

La petición entra por los middlewares, la ruta la envía a una vista, la vista usa los modelos para leer o escribir en SQL Server y una plantilla arma la página de respuesta.

App	Responsabilidad
catalogos	Datos de referencia: países, departamentos, municipios, estados, deportes, ligas, equipos, tipos de mercado y métodos de pago

App	Responsabilidad
cuentas	Credenciales, usuarios, administradores, super administradores, recuperación de contraseña y bitácoras de acceso
billetera	Saldo del usuario, transacciones, cuentas bancarias, solicitudes de retiro e historial de saldo
apuestas	Eventos deportivos, resultados, mercados, opciones con cuota, promociones, boletos y sus historiales
auditoria	Bitácora de eventos de seguridad y el middleware de seguridad perimetral
panel_publico	Su models.py no tiene modelos; su vista de inicio muestra los eventos de hoy y los resultados de ayer

3. Tecnologías back-end

El back-end es Python con Django, conectado a SQL Server mediante el ORM de Django y un driver ODBC. Cada pieza cumple un papel concreto:

Tecnología	Para qué se usa en el proyecto	Dónde se configura
Python + Django	Framework web: rutas, vistas, plantillas, panel de administración, autenticación y sesiones	settings.py
Django ORM	Define las tablas como clases Python (modelos) y consulta la base sin escribir SQL a mano	models.py de cada app
SQL Server	Motor de base de datos relacional donde se guardan todos los datos (SQL Server 2025 en desarrollo)	DATABASES en settings.py
Motor mssql	Backend de Django que traduce el ORM a SQL	ENGINE: 'mssql'

Tecnología	Para qué se usa en el proyecto	Dónde se configura
	Server (paquete mssql-django; versiones en la tabla de librerías, tomadas de pip freeze)	
ODBC Driver 18 for SQL Server	Driver de Microsoft que abre la conexión desde Python; se usa con TrustServerCertificate=yes	OPTIONS en DATABASES
python-decouple	Lee la configuración sensible (llaves, credenciales) desde variables de entorno o el archivo .env	config(...) y Csv en settings.py
Plantillas de Django	Generan el HTML de las pantallas, en la carpeta templates/ y dentro de cada app	TEMPLATES
Hashers de Django	Guardan contraseñas y respuestas de seguridad cifradas con make_password y check_password	cuentas/models.py
Sesiones y CSRF de Django	Mantienen el inicio de sesión (4 horas) y protegen los formularios	Sección de cookies en settings.py
Envío de correo (SMTP)	Envía el enlace de recuperación de contraseña; por defecto imprime el correo en consola	EMAIL_* en settings.py
Archivos estáticos y media	Sirven CSS, JS e imágenes desde static/ y media/	STATIC_* y MEDIA_*

Las migraciones fueron generadas con Django 4.2.30. Otros valores generales: idioma es-gt, zona horaria America/Guatemala, fechas con zona horaria activada (USE_TZ = True) y llaves primarias BigAutoField por defecto.

Librerías instaladas

El resultado de `pip freeze` muestra 13 paquetes. Solo tres los eligió el proyecto; el resto llega como dependencia.

Librería	Versión	Para qué sirve
Django	4.2.30	Framework web
mssql-django	1.8.0	Motor mssql de Django para SQL Server
python-decouple	3.8	Lectura de las variables del .env
pyodbc	5.3.0	Conexión ODBC desde Python; la usa mssql-django
pytz	2026.4	Zonas horarias; la usa mssql-django
asgiref, sqlparse, tzdata	3.12.1, 0.6.0, 2026.4	Dependencias de Django
pillow	12.1.1	Manejo de imágenes; en el código revisado no hay campos de imagen que la necesiten
virtualenv, distlib, filelock, platformdirs, python-discovery	21.7.8, 0.4.3, 3.32.5, 4.11.7, 1.6.0	Herramientas para crear entornos virtuales; no son parte del proyecto

El comando se ejecutó sin el prefijo (venv), por lo que la lista parece venir de la instalación global de Python y no del entorno virtual del proyecto. Un requirements.txt mínimo sería:

```
Django==4.2.30
mssql-django==1.8.0
python-decouple==3.8
```

4. Base de datos

El proyecto usa una única base de datos SQL Server, definida como conexión default. Django la consulta a través de su ORM y los datos de acceso no están escritos en el código: se leen de variables de entorno.

Parámetro	Valor	Origen
Motor	mssql (SQL Server)	Fijo en settings.py

Parámetro	Valor	Origen
Nombre de la base	DB_NAME	Variable de entorno
Usuario	DB_USER	Variable de entorno
Contraseña	DB_PASSWORD	Variable de entorno
Servidor	DB_HOST	Variable de entorno
Puerto	DB_PORT	Variable de entorno
Driver	ODBC Driver 18 for SQL Server	Fijo en settings.py
Parámetro extra	TrustServerCertificate=yes	Fijo en settings.py

Cómo se usa la base de datos

- **Modelos = tablas.** Cada clase de models.py es una tabla. El nombre real se fija con db_table y lleva el prefijo tbl_ (por ejemplo tbl_boleto, tbl_billetera).
- **Nombres de columna propios.** Las llaves foráneas usan db_column para que la columna se llame igual que en el script SQL (por ejemplo id_usuario).
- **Integridad referencial.** Las relaciones usan on_delete=PROTECT en catálogos y registros financieros, para impedir borrar datos que otros usan, y CASCADE en bitácoras y detalles que dependen de su registro principal.
- **Validaciones en el modelo.** Los montos y cuotas llevan MinValueValidator (por ejemplo cuota mínima de 1.01 y monto mínimo apostado de 1) y los importes usan DecimalField con 2 decimales.
- **Restricciones de unicidad.** Se usan unique_together para evitar duplicados, por ejemplo una opción repetida en un mismo boleto o dos mercados iguales en un mismo evento.
- **Índices.** Hay índices sobre fecha_inicio del evento, sobre ip_origen y fechas en intentos de login y en la bitácora de seguridad.
- **Script SQL.** El archivo sql/bd_apuestas_deportivas.sql contiene el script de la base. Crea la base bd_apuestas_deportivas si no existe y las 38 tablas tbl_*, convertidas desde un script de MySQL. También define reglas en la base: restricciones CHECK para estados y valores no negativos, llaves únicas, índices y llaves foráneas, con borrado en cascada en bitácoras y detalles. Las fechas del script usan la hora del servidor SQL, pero Django llena las suyas con su propio reloj.

Migraciones. Las cinco apps con modelos (catalogos, cuentas, billetera, apuestas y auditoria) tienen su 0001_initial.py, generado con Django 4.2.30 el 10 de septiembre de 2026; panel_publico no tiene modelos. Crean las mismas tablas tbl_* y los mismos nombres de columna que el script SQL, pero no incluyen las restricciones CHECK ni el borrado en cascada que define el script.

5. Modelo de datos

La base tiene 38 tablas propias del proyecto, repartidas en cinco apps: 9 de catálogos, 12 de cuentas, 5 de billetera, 11 de apuestas y 1 de auditoría. Todas llevan el prefijo tbl_.



relaciones principales · 10 tablas

La cadena va de la cuenta al partido: un login tiene un usuario, el usuario una billetera y la billetera varios boletos. Cada boleto se desglosa en detalles que apuntan a opciones de apuesta, y cada opción pertenece a un mercado de un evento deportivo.

Catálogos (catalogos)

Tabla	Qué guarda	Relaciones clave
tbl_pais	Países y su código ISO	Base de departamentos y ligas
tbl_departamento	Departamentos de cada país	Pertenece a un país
tbl_municipio	Municipios de cada departamento	Pertenece a un departamento
tbl_estado	Estados genéricos (activo, inactivo, bloqueado, suspendido)	Lo usan usuarios, administradores, billeteras y deportes
tbl_deporte	Deportes	Tiene un estado

Tabla	Qué guarda	Relaciones clave
tbl_liga	Ligas	Pertenece a un deporte y a un país
tbl_equipo_atleta	Equipos o atletas	Pertenece a un deporte
tbl_tipo_mercado	Tipos de mercado de apuesta	Lo usan los mercados de evento
tbl_metodo_pago	Métodos de pago y su comisión en porcentaje	Lo usan las transacciones

Cuentas (cuentas)

Tabla	Qué guarda	Relaciones clave
tbl_login	Credenciales únicas, tipo de cuenta (usuario, administrador o super administrador), intentos fallidos y bloqueo	Raíz de usuario, administrador y super administrador
tbl_login_log	Registro de accesos con la IP	Pertenece a un login
tbl_usuario	Datos personales, contacto y ubicación del jugador	1 a 1 con login; departamento, municipio y estado
tbl_administrador	Empleados con rol: operador de eventos, cajero o auditor	1 a 1 con login; estado
tbl_super_administrador	Super administrador y su llave de doble factor	1 a 1 con login
tbl_pregunta_seguridad	Catálogo de preguntas para recuperar la contraseña	Lo usan las respuestas de seguridad
tbl_usuario_pregunta_seguridad	Respuestas cifradas de cada login	Login y pregunta, únicos por par
tbl_recuperacion_password	Token temporal, expiración y si ya se usó	Pertenece a un login
tbl_admin_log	Acciones de administración y tabla	Pertenece a un login

Tabla	Qué guarda	Relaciones clave
	afectada	
tbl_historial_sesiones	Sesiones con IP, dispositivo y resultado	Pertenece a un login
tbl_historial_bloqueos_cuenta	Bloqueos con motivo, quién los pidió y fechas	Pertenece a un usuario
tbl_intento_login	Intentos de acceso por IP, exitosos o fallidos	Guarda el identificador como texto, sin llave foránea

Billetera (billetera)

Tabla	Qué guarda	Relaciones clave
tbl_billetera	Saldo disponible y saldo retenido; el saldo total es la suma de ambos	1 a 1 con usuario; estado
tbl_transaccion	Depósitos, retiros, débitos por apuesta, premios y ajustes	Billetera; método de pago opcional
tbl_cuenta_bancaria_usuario	Cuentas bancarias del usuario, monetaria o de ahorro	Pertenece a un usuario
tbl_solicitud_retiro	Retiros con estado: pendiente, aprobada, rechazada o pagada	Billetera y cuenta bancaria
tbl_historial_saldo_billetera	Cada movimiento con saldo anterior, saldo nuevo y tipo de operación	Pertenece a una billetera

Apuestas (apuestas)

Tabla	Qué guarda	Relaciones clave
tbl_evento	Evento con fecha de inicio y estado: programado, en juego, finalizado, suspendido o cancelado	Liga, equipo local y equipo visitante
tbl_resultado_evento	Marcador final y	1 a 1 con evento

Tabla	Qué guarda	Relaciones clave
	observaciones	
tbl_mercado_evento	Mercado con estado: abierto, cerrado o liquidado	Evento y tipo de mercado, únicos por par
tbl_opcion_apuesta	Opción con su cuota actual y marca de ganadora	Pertenece a un mercado
tbl_promocion_cuota	Aumento porcentual de una cuota durante un periodo	Pertenece a una opción
tbl_boleto	Monto apostado, cuota total, ganancia potencial y estado: pendiente, ganado, perdido, anulado o pagado	Pertenece a una billetera
tbl_detalle_boleto	Opciones elegidas con la cuota del momento de la apuesta	Boleto y opción, únicos por par
tbl_historial_cuotas	Cambios de cuota, con el administrador que los hizo	Opción y administrador
tbl_historial_estado_boleto	Cambios de estado de un boleto y su motivo	Pertenece a un boleto
tbl_historial_cambios_evento	Cambios de estado con minuto del partido y marcador	Pertenece a un evento
tbl_historial_jugador_resumen	Resumen mensual: boletos, monto apostado, monto reclamado y rendimiento neto	Usuario y mes, únicos por par

Un boleto es compuesto cuando tiene más de un detalle, es decir, una apuesta combinada.

Auditoría (auditoria)

Una sola tabla, tbl_bitacora_seguridad, guarda los eventos de seguridad con tipo, IP de origen, ruta solicitada, detalle y si la petición fue bloqueada. Se describe en la sección 6.

6. Seguridad

La seguridad se apoya en dos middlewares propios, en parámetros configurables y en tablas de bitácora dentro de la base de datos.

Middlewares propios (los últimos de la lista MIDDLEWARE):

- auditoria.middleware.SeguridadPerimetralMiddleware: limita las peticiones por IP (60 por minuto, con la caché de Django) y responde con error 429 al excederlas; luego revisa los valores GET y POST contra patrones de inyección SQL y de <script>, y responde con error 403 si encuentra uno. Cada bloqueo se guarda en tbl_bitacora_seguridad; si ese registro falla, la petición no se interrumpe.
- cuentas.middleware.ControlAccesoMiddleware: segunda capa de control por prefijo de ruta. /panel-administrativo/ exige sesión de administrador o super administrador y /panel-usuario/ exige sesión de usuario; si no, redirige al login que corresponde.

Parámetros de seguridad en settings.py

Parámetro	Valor	Efecto
SEGURIDAD_MAX_INTENTOS_LOGIN	5	Intentos fallidos antes de bloquear la cuenta
SEGURIDAD_MINUTOS_BLOQUEO_LOGIN	15	Duración del bloqueo temporal
SEGURIDAD_MAX_PETICIONES_POR_MINUTO	60	Límite de peticiones por IP (anti-DDOS)
SEGURIDAD_MINUTOS_VIGENCIA_TOKEN_RESET	30	Vigencia del enlace de recuperación de contraseña
SESSION_COOKIE_AGE	4 horas	Duración de la sesión; al iniciar sesión se fija en 4 horas, aunque se cierre el navegador

Contraseñas y sesiones

- Las contraseñas se guardan con hash mediante make_password, nunca en texto plano. Los formularios exigen al menos 8 caracteres y una confirmación. Los validadores de AUTH_PASSWORD_VALIDATORS están definidos en settings.py, pero el código revisado no los invoca, así que hoy no se aplican.
- Las respuestas de las preguntas de seguridad también se guardan con hash, normalizadas a minúsculas y sin espacios sobrantes.
- Las cookies de sesión son HttpOnly y SameSite=Lax; la protección CSRF está activa y X_FRAME_OPTIONS es DENY.

- DJANGO_SSL_REDIRECT activa la redirección a HTTPS y las cookies seguras; por defecto está apagado, pensado para desarrollo.

Tablas de control y bitácora

Tabla	Qué registra
tbl_login	Intentos fallidos, fecha de bloqueo (bloqueado_hasta) y último cambio de contraseña
tbl_login_log e tbl_historial_sesiones	Accesos con IP, dispositivo y resultado (exitoso o fallido)
tbl_intento_login	Intentos de acceso por IP, para frenar fuerza bruta
tbl_historial_bloqueos_cuenta	Bloqueos de cuenta, su motivo y quién los pidió (sistema, administrador o usuario)
tbl_recuperacion_password	Tokens UUID de recuperación con fecha de expiración y marca de uso
tbl_admin_log	Acciones de administración y la tabla afectada
tbl_bitacora_seguridad	Eventos de seguridad: inyección SQL, DDOS, valores inválidos, límite excedido y acceso no autorizado

Los administradores tienen tres roles estrictos: operador de eventos, cajero y auditor. El super administrador guarda una llave secreta para autenticación de dos factores (llave_mfa_secret).

7. Configuración y variables de entorno

El proyecto lee 16 variables con python-decouple, desde el archivo .env en la raíz o desde el sistema. Las cinco de base de datos no tienen valor por defecto, así que el proyecto no arranca sin ellas.

Variable	Obligatoria	Valor por defecto	Para qué sirve
DJANGO_SECRET_KEY	En producción	Llave de ejemplo insegura	Firma sesiones y tokens
DJANGO_DEBUG	No	True	Modo depuración; debe ser False en producción

Variable	Obligatoria	Valor por defecto	Para qué sirve
DJANGO_ALLOWED_HOSTS	No	127.0.0.1,localhost	Dominios permitidos, separados por comas
DB_NAME	Sí	—	Nombre de la base de datos
DB_USER	Sí	—	Usuario de SQL Server
DB_PASSWORD	Sí	—	Contraseña de SQL Server
DB_HOST	Sí	—	Servidor de SQL Server
DB_PORT	Sí	—	Puerto de SQL Server
DJANGO_SSL_REDIRECT	No	False	Fuerza HTTPS y activa cookies seguras
EMAIL_BACKEND	No	Consola	Dónde se envían los correos
EMAIL_HOST	No	smtp.gmail.com	Servidor SMTP
EMAIL_PORT	No	587	Puerto SMTP
EMAIL_USE_TLS	No	True	Cifrado TLS en el correo
EMAIL_HOST_USER	No	vacío	Usuario del correo
EMAIL_HOST_PASSWORD	No	vacío	Contraseña del correo
DEFAULT_FROM_EMAIL	No	no-responder@apuestasdeportivas.local	Remitente de los correos

Valores en el entorno de desarrollo. No se incluyen la llave secreta ni las contraseñas.

Variable	Valor actual
DJANGO_DEBUG	True
DJANGO_ALLOWED_HOSTS	127.0.0.1,localhost
DB_NAME	bd_apuestas_deportivas
DB_HOST	127.0.0.1, es decir, SQL Server 2025 en la misma máquina
DB_PORT	1433
EMAIL_BACKEND	Consola: los correos se imprimen en la terminal
EMAIL_HOST_USER y EMAIL_HOST_PASSWORD	Vacíos
DJANGO_SSL_REDIRECT	False

Recomendaciones

- No subir el archivo .env al repositorio ni compartirlo; contiene credenciales. Conviene mantener un .env.example solo con los nombres de las variables.
- En producción, definir una DJANGO_SECRET_KEY propia, poner DJANGO_DEBUG=False, ajustar DJANGO_ALLOWED_HOSTS y activar DJANGO_SSL_REDIRECT=True.
- Para que el correo de recuperación de contraseña salga de verdad, cambiar EMAIL_BACKEND a django.core.mail.backends.smtp.EmailBackend y llenar las variables de usuario y contraseña del correo.
- El .env actual conecta con la cuenta administradora de SQL Server. En producción conviene un usuario con solo los permisos que la aplicación necesita.
- El archivo empieza con una nota que pide copiarlo como .env, y settings.py menciona un .env.example. Conviene guardar un .env.example con los nombres de las variables y sin valores, y agregar .env al .gitignore.

8. Instalación y ejecución

Pasos para levantar el proyecto en Windows, desde la carpeta que contiene manage.py:

1. Instalar Python, SQL Server y el controlador **ODBC Driver 18 for SQL Server**.
2. Crear y activar el entorno virtual: python -m venv venv y luego venv\Scripts\activate.
3. Instalar las librerías con pip install -r requirements.txt. Se puede crear con el contenido mínimo de la sección 3 o generar con pip freeze > requirements.txt en el entorno actual.

4. Crear el archivo .env con las variables de la sección 7, al menos las cinco DB_*.
5. Crear la base bd_apuestas_deportivas y sus 38 tablas ejecutando en SQL Server el script sql/bd_apuestas_deportivas.sql.
6. Ejecutar python manage.py migrate con la opción --fake-initial, para crear las tablas propias de Django (sesiones, administración, autenticación) sin recrear las del script.
7. Iniciar el servidor con python manage.py runserver y abrir http://127.0.0.1:8000.

Por confirmar: los modelos no usan managed = False, así que migrate también intentará crear las tablas tbl_* que ya crea el script SQL. Hay que decidir si la base se arma solo con el script o solo con migraciones, y dejarlo escrito aquí.

Opción sugerida: ejecutar el script SQL y luego python manage.py migrate --fake-initial. El script no crea las tablas de Django, y las sesiones se guardan en la base, así que migrate sigue siendo necesario. Con --fake-initial, Django marca como aplicadas las migraciones iniciales de las apps cuyas tablas ya existen. Esto supone que cada app tiene su carpeta migrations/. Las cinco apps con modelos la tienen. Si se prefiere una sola vía, el script SQL es más completo, porque las migraciones no crean sus restricciones CHECK ni su borrado en cascada.

9. Rutas y lógica de negocio

El proyecto tiene 22 rutas. Las de cuentas llevan el prefijo /cuentas/; las demás cuelgan de la raíz, y los paneles se separan por prefijo: /panel-usuario/ y /panel-administrativo/. En modo depuración también se sirven los archivos de media/.

Rutas

Ruta	Vista	Acceso	Qué hace
/	panel_publico.inicio	Público	Muestra los eventos de hoy con sus mercados y opciones, y los resultados de ayer
/admin/	Django admin	Cuentas de Django	Panel de administración estándar de Django
/cuentas/registro/	registro_usuario	Público	Registro de usuario final
/cuentas/api/municipios/<id_departamento>/	api_municipios	Público	Devuelve en JSON los municipios de un departamento

Ruta	Vista	Acceso	Qué hace
/cuentas/login/	login_usuario	Público	Inicio de sesión de usuarios
/cuentas/login-admin/	login_admin	Público	Inicio de sesión de administradores y super administradores
/cuentas/logout/	logout_view	Con sesión	Cierra la sesión y vuelve al inicio
/cuentas/recuperar/	recuperar_password_solicitud	Público	Paso 1 de la recuperación: usuario y correo registrado
/cuentas/recuperar/pregunta/	recuperar_password_pregunta	Público	Paso 2: pregunta de seguridad
/cuentas/recuperar/confirmar/<token>/	recuperar_password_confirmar	Con enlace del correo	Paso 3: nueva contraseña
/panel-usuario/	panel_usuario_inicio	Usuario	Saldo y hasta 30 eventos programados próximos
/panel-usuario/crear-boleto/	crear_boleto	Usuario	Crea un boleto (solo acepta POST)
/panel-usuario/mis-boletos/	mis_boletos	Usuario	Lista sus boletos, del más reciente al más antiguo
/panel-usuario/deposito/	deposito	Usuario	Acredita un depósito a la billetera
/panel-usuario/cuentas-bancarias/	mis_cuentas_bancarias	Usuario	Lista y agrega cuentas bancarias
/panel-usuario/retiro/	solicitar_retiro	Usuario	Solicita un retiro y ve el estado de sus solicitudes

Ruta	Vista	Acceso	Qué hace
/panel-administrativo/	panel_admin_inicio	Administrador o super administrador	Contadores: usuarios, eventos programados, boletos pendientes y retiros pendientes
/panel-administrativo/eventos/	admin_eventos	Operador de eventos o super administrador	Crea y lista eventos
/panel-administrativo/eventos/<id_evento>/mercados/	admin_mercados_evento	Operador de eventos o super administrador	Agrega mercados y opciones con su cuota
/panel-administrativo/opciones/<id_opcion>/actualizar-cuota/	admin_actualizar_cuota	Operador de eventos o super administrador	Cambia la cuota de una opción
/panel-administrativo/eventos/<id_evento>/liquidar/	admin_liquidar_evento	Operador de eventos o super administrador	Registra el resultado y liquida los boletos
/panel-administrativo/retiros/	admin_solicitudes_retiro	Cajero o super administrador	Aprueba o rechaza solicitudes de retiro

Las apps catalogos y auditoria no tienen vistas propias. Los permisos se aplican en tres capas: el middleware por prefijo de ruta, el decorador requiere_sesion con los tipos de cuenta permitidos y, en las vistas administrativas, la comprobación del rol.

Crear un boleto

1. Valida el formulario con el monto y las opciones elegidas.
2. Comprueba que cada opción exista, que su mercado esté abierto y que su evento esté programado.
3. Calcula la cuota total multiplicando las cuotas de las opciones y la ganancia potencial como monto por cuota total, ambas con 2 decimales.
4. Dentro de una transacción, bloquea la fila de la billetera y verifica que el saldo disponible alcance.
5. Mueve el monto de saldo disponible a saldo retenido y registra el movimiento de débito por apuesta en el historial de saldo.

6. Crea el boleto en estado pendiente y un detalle por opción, guardando la cuota del momento.

Liquidar un evento

1. El operador registra el marcador y las opciones ganadoras; el evento pasa a finalizado y el cambio queda en el historial del evento con el marcador.
2. Las opciones se marcan como ganadoras o no, y todos los mercados del evento pasan a liquidado.
3. Cada boleto pendiente con una selección de ese evento solo se liquida si todos los eventos del boleto ya finalizaron.
4. Si todas sus opciones ganaron, el boleto pasa a ganado: se descuenta el monto del saldo retenido y se acredita la ganancia potencial al saldo disponible.
5. Si alguna opción perdió, el boleto pasa a perdido y el monto se descuenta del saldo retenido.
6. Cada cambio de estado se guarda en el historial del boleto con el motivo "Liquidación automática de evento".

Depósitos y retiros

- **Depósito:** suma el monto al saldo disponible al instante y registra una transacción de depósito y su historial de saldo. En el código revisado no hay pasarela de pago.
- **Solicitud de retiro:** el usuario elige una de sus cuentas bancarias y un monto no mayor a su saldo disponible; la solicitud queda pendiente.
- **Gestión del cajero:** al aprobar, descuenta el saldo disponible, crea la transacción de retiro y su historial, y marca la solicitud como pagada; al rechazar, solo cambia su estado. No existe un paso intermedio de "aprobada".

Registro, sesión y recuperación de contraseña

- **Registro:** en una sola transacción crea el login con contraseña cifrada, el usuario, su respuesta de seguridad y su billetera. Los estados `ACTIVO` y `ACTIVA` se crean en `tbl_estado` si no existen.
- **Inicio de sesión:** `login_usuario` y `login_admin` comparten la misma lógica. Busca el login por nombre de usuario y tipo permitido, revisa si la cuenta está bloqueada, verifica el hash y cuenta los intentos fallidos. Cada intento se guarda en `tbl_intento_login` y cada sesión en `tbl_historial_sesiones`.
- **Recuperación:** primero se piden usuario y correo registrado, con una respuesta neutra que no revela si el usuario existe; luego la pregunta de seguridad; si se acierta, se crea un token UUID válido por 30 minutos y se envía el enlace por correo; por último se define la nueva contraseña y el token queda marcado como usado.

Validaciones de los formularios

Formulario	Reglas
Registro	Mayor de 18 años; teléfono de 8 a 20 caracteres con números, +, - y espacios; correo único; usuario único de 4 a 50 caracteres con letras, números, punto y guion bajo; contraseña de mínimo 8 con confirmación; pregunta y respuesta de seguridad
Inicio de sesión	Usuario y contraseña
Recuperación	Usuario y correo; respuesta de seguridad; nueva contraseña de mínimo 8 con confirmación
Boleto	Monto mayor a 0 y hasta 999,999.99; opciones como ids separados por coma, al menos una y sin repetir
Evento	Equipo local distinto del visitante
Mercado y opción	Cuota mayor a 1.00
Resultado	Marcadores enteros no negativos
Depósito	Monto mayor a 0 y hasta 50,000 por transacción
Retiro	Monto mayor a 0; el saldo se comprueba en la vista
Cuenta bancaria	Banco, número de cuenta y tipo (monetaria o ahorro)

Sesión y plantillas

- **Sesión propia:** el proyecto no usa `django.contrib.auth.User` para sus usuarios, sino `tbl_login`. `iniciar_sesion` limpia la sesión anterior y guarda el id del login, el tipo de cuenta y el nombre a mostrar; la sesión dura 4 horas.
- **Bloqueo de cuenta:** al llegar a 5 intentos fallidos se guarda `bloqueado_hasta` 15 minutos adelante y el contador vuelve a 0. Un inicio de sesión correcto limpia ambos valores.
- **Decorador `requiere_sesion`:** recibe los tipos de cuenta permitidos y, si no coinciden, redirige al login de administradores cuando la lista incluye `ADMIN` o `SUPER_ADMIN`, y al de usuarios en los demás casos.

- **Procesador de contexto `sesion_actual`:** pone el id, el tipo, el nombre y un indicador de sesión activa a disposición de todas las plantillas.
- **Plantillas:** las pantallas de `cuentas` heredan de `base.html` y usan los bloques `titulo` y `contenido`. Cada formulario recorre sus campos, muestra los errores de cada uno e incluye el token CSRF. El registro llena la lista de municipios con una petición `fetch` a la ruta `/cuentas/api/municipios/`.
- **Panel de Django:** `/admin/` usa las cuentas propias de Django, separadas de `tbl_login`.

Pantallas

Todas las plantillas heredan de `base.html`, que arma el encabezado con un menú distinto para visitantes, usuarios y administradores, muestra los mensajes de Django y carga `css/estilos.css`.

Plantilla	Vista	Qué muestra
`panel_publico/inicio.html`	`inicio`	Eventos de hoy con sus cuotas y tabla de resultados de ayer
`cuentas/login_usuario.html` y `login_admin.html`	`login_usuario`, `login_admin`	Formularios de acceso con enlace para recuperar la contraseña
`cuentas/registro.html`	`registro_usuario`	Formulario de registro; los municipios se cargan con JavaScript al elegir departamento
`cuentas/recuperar_*.html`	Vistas de recuperación	Los tres pasos: usuario y correo, pregunta de seguridad y nueva contraseña
`apuestas/panel_usuario_inicio.html`	`panel_usuario_inicio`	Saldos, eventos disponibles y el boleto, que se arma en pantalla con JavaScript: suma las cuotas, estima la ganancia y envía los ids de las opciones separados por coma
`apuestas/mis_boletos.html`	`mis_boletos`	Tabla de boletos con selecciones, monto, cuota total, ganancia potencial y

Plantilla	Vista	Qué muestra
		estado
billetera/deposito.html, mis_cuentas_bancarias.html, solicitar_retiro.html	deposito, mis_cuentas_bancarias, solicitar_retiro	Depósito, cuentas bancarias y retiros con su historial
apuestas/_panel_admin_base.html	Panel administrativo	Menú lateral: resumen, eventos y mercados, retiros y enlace a /admin/
apuestas/panel_admin_inicio.html	panel_admin_inicio	Rol actual y cuatro contadores
apuestas/admin_eventos.html	admin_eventos	Formulario para crear eventos y tabla con los botones de mercados y de liquidar (este solo en eventos programados)
apuestas/admin_mercados_evento.html	admin_mercados_evento	Alta de mercados y opciones, y actualización de cuotas
apuestas/admin_liquidar_evento.html	admin_liquidar_evento	Marcador y opciones ganadoras por mercado, con confirmación antes de pagar
billetera/admin_solicitudes_retiro.html	admin_solicitudes_retiro	Solicitudes con botones de aprobar y rechazar

Aspectos a considerar

- **IP del cliente:** los middlewares y las vistas toman la IP de la cabecera X-Forwarded-For cuando existe, y un cliente puede falsificarla si no hay un proxy de confianza delante.
- **Filtro anti inyección SQL:** revisa los valores GET y POST, no las cabeceras, aunque el comentario del código las menciona. Rechaza cualquier valor con comilla simple, doble guion o #, por lo que puede bloquear datos legítimos como direcciones o contraseñas con esos caracteres.
- **Límite de peticiones:** usa la caché de Django y settings.py no define CACHES. Por defecto Django usa memoria local de cada proceso, así que con varios procesos el conteo no se comparte.

- **Transacciones:** los tipos de transacción de apuesta y de premio existen en tbl_transaccion, pero las vistas revisadas solo crean depósitos y retiros; las apuestas y premios quedan en el historial de saldo.
- **Tablas sin uso en las vistas revisadas:** tbl_promocion_cuota y tbl_historial_jugador_resumen.
- **Rol de auditor:** en lo revisado solo puede entrar al panel de inicio administrativo; ninguna vista le asigna funciones propias.
- **Más tablas sin uso:** en el código revisado no se escribe en tbl_historial_bloqueos_cuenta, tbl_admin_log ni tbl_login_log; los accesos se guardan en tbl_historial_sesiones. De los tipos de evento de tbl_bitacora_seguridad, los middlewares solo registran SQLI y DDOS.
- **Identificadores sin validar:** los formularios de evento, mercado y depósito reciben ids como números y no comprueban que existan; un id inexistente provocaría un error de base de datos.
- **Cierre de apuestas:** crear_boleto solo comprueba que el evento esté programado y el mercado abierto; no compara la hora de inicio, así que se puede apostar después de la hora si el evento sigue programado. Ninguna vista pasa un evento a en juego, suspendido o cancelado, ni cierra un mercado.
- **Liquidación:** el botón de liquidar aparece solo en eventos programados, pero la vista no comprueba el estado del evento.
- **Cuota al confirmar:** el boleto se calcula en el servidor con la cuota vigente al confirmar, que puede diferir de la que el usuario vio. La cuota total y la ganancia que muestra la página son solo estimaciones.